

UD 2. Introducción a C#

2º CFGS DESARROLLO APLICACIONES MULTIPLATAFORMA
MÓDULO DESARROLLO DE INTERFACES

ÍNDICE

<i>Índice</i>	<i>1</i>
1. Características de C#	4
2. Descarga Visual Studio Community.....	5
3. Creación de un proyecto en c#	6
3.1 Hola Mundo	8
3.2 Sintaxis	9
4. Elementos de un programa en C#	10
4.1 Comentarios.....	10
4.2 Variables	11
4.2.1 Constantes	12
4.3 Tipos de datos	13
4.4 Entrada de datos	13
4.5 Operadores	15
4.5.1 Operadores aritméticos.....	15
4.5.2 Operadores de asignación	16
4.5.3 Operadores de comparación	16
4.5.4 Operadores lógicos.....	17
4.6 Clase Math	17
4.7 Strings.....	18
4.8 Booleanos	18
4.9 Formato de salida.....	19

4.10	Arrays	20
5.	Estructuras iterativas	22
5.1	If...Else	22
5.2	Switch	24
5.3	While	26
5.3.1	Bucle While	26
5.3.2	Bucle Do/While.....	26
5.4	For	27
5.5	Break y continue.....	28
6.	Métodos	29
6.1	Creación de métodos	29
6.2	Llamada a métodos	29
6.3	Parámetros	30
6.4	Sobrecarga de métodos	32
7.	Clases.....	32
7.1	Crear una clase	33
7.2	Crear un objeto	33
7.3	Uso de múltiples clases	34
7.4	Miembros de una clase	35
7.5	Constructores	37
7.6	Modificadores de acceso.....	38
7.7	Propiedades	39
7.8	Herencia.....	40

7.9	Polimorfismo.....	41
7.10	Archivos	44
7.11	Excepciones.....	45

1. CARACTERÍSTICAS DE C#

C # se pronuncia "C-Sharp". Es un lenguaje de programación orientado a objetos creado por Microsoft que se ejecuta en .NET Framework.

C # tiene raíces de la familia C y el lenguaje está cerca de otros lenguajes populares como [C ++](#) y [Java](#) .

La primera versión se lanzó en el año 2002. La última versión, **C# 11** , se lanzó en marzo de 2023.

C # se usa para:

- Aplicaciones móviles
- Aplicaciones de escritorio
- aplicaciones web
- servicios web
- Sitios web
- Juegos
- VR
- Aplicaciones de base de datos
- ¡Y mucho, mucho más!

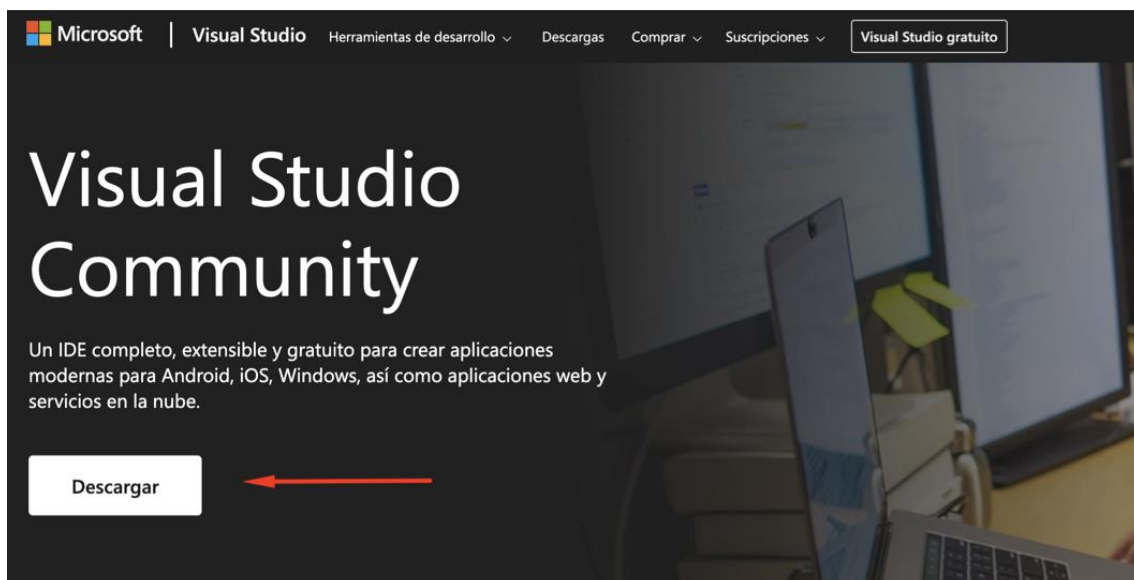
¿POR QUÉ USAR C#?

- Es uno de los lenguajes de programación más populares del mundo.
- Es fácil de aprender y fácil de usar.
- Tiene un gran apoyo de la comunidad.
- C# es un lenguaje orientado a objetos que brinda una estructura clara a los programas y permite que el código se reutilice, lo que reduce los costos de desarrollo.
- Como C # está cerca de C, [C ++](#) y [Java](#) , facilita a los programadores cambiar a C # o viceversa.

2.DESCARGA VISUAL STUDIO COMMUNITY

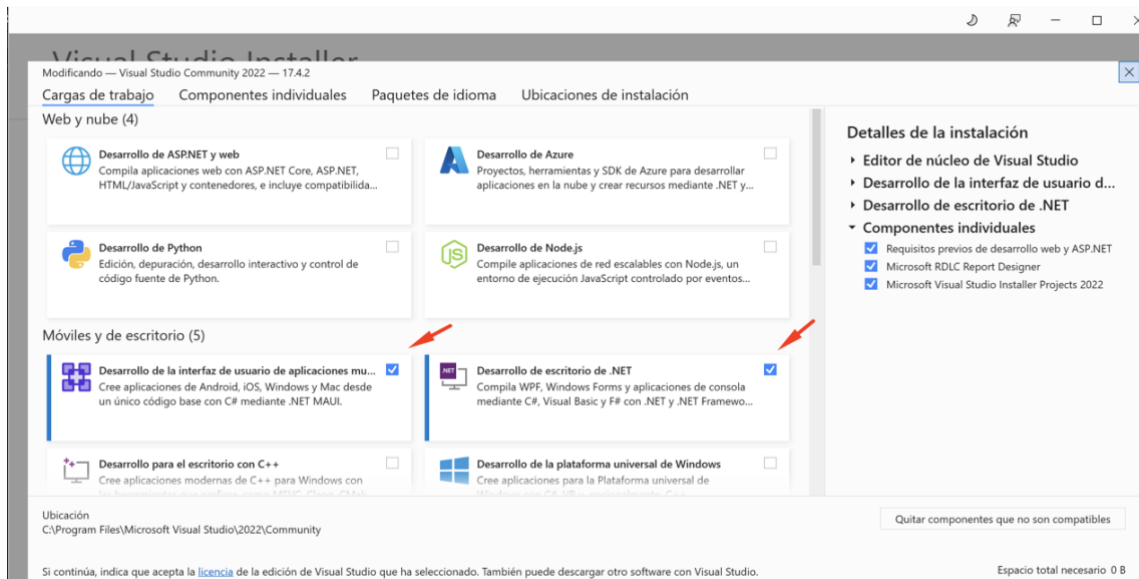
Podemos utilizar el Visual Studio Community 2022 que tiene entre otros lenguajes el C# y lo podemos descargar desde aquí:

<https://visualstudio.microsoft.com/es/vs/community/>



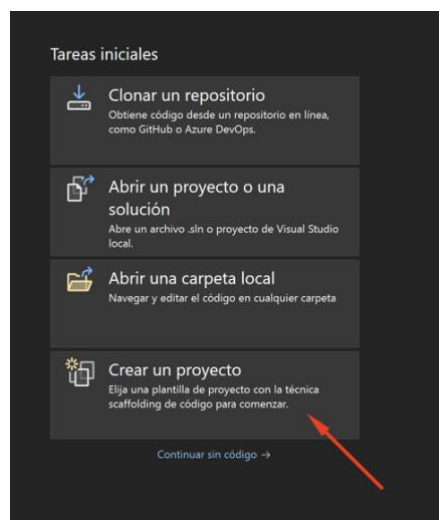
A la hora de instalarlo hay que comprobar que se descarguen las siguientes “Cargas de trabajo”, aunque esto también se puede descargar a posteriori:

UD 2. INTRODUCCIÓN A C#



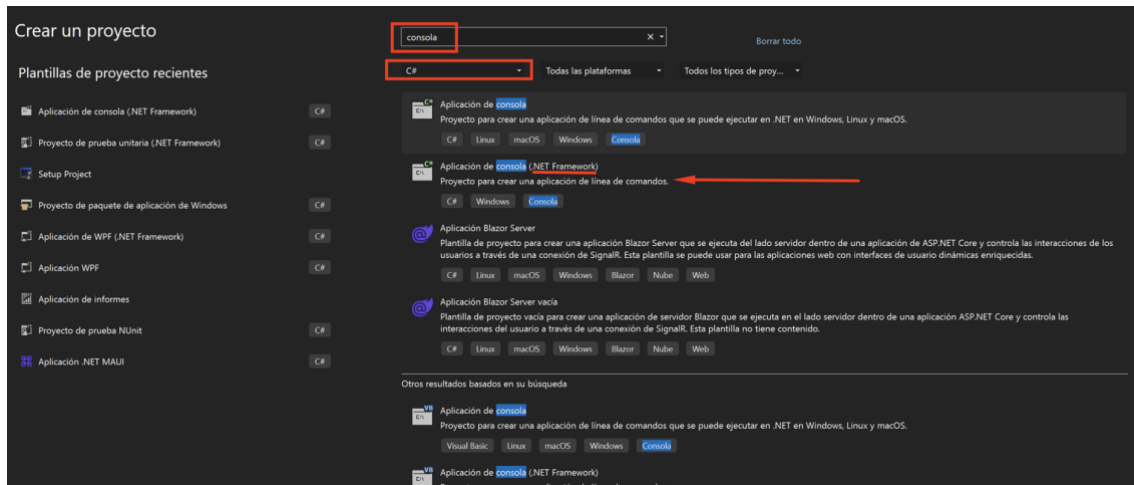
3. CREACIÓN DE UN PROYECTO EN C#

Para crear un nuevo proyecto en C# basta con elegir la correspondiente opción en la pantalla de inicio o en el menú superior de la aplicación.



UD 2. INTRODUCCIÓN A C#

El primer proyecto será un proyecto de consola. Elegimos el siguiente tipo:



A continuación pedirá los datos para el nuevo proyecto. Es recomendable realizar el programa con el framework 4.7.2:

Configure su nuevo proyecto

Aplicación de consola (.NET Framework) C# Windows Consola

Nombre del proyecto

2DAM_PrimerAplicacionConsola

Ubicación

C:\Users\dan\source\repos

Nombre de la solución ⓘ

2DAM_PrimerAplicacionConsola

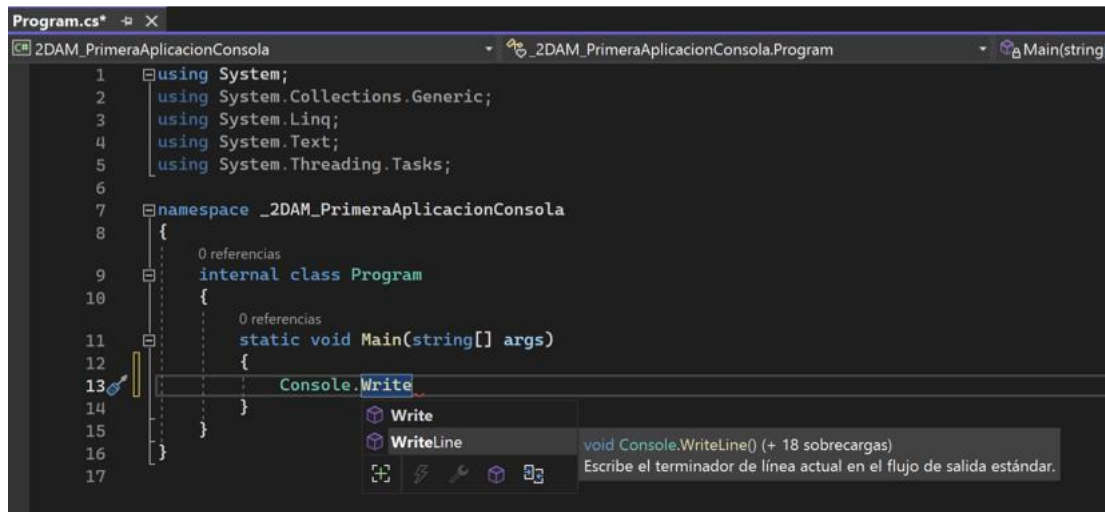
☐ Colocar la solución y el proyecto en el mismo directorio

Marco

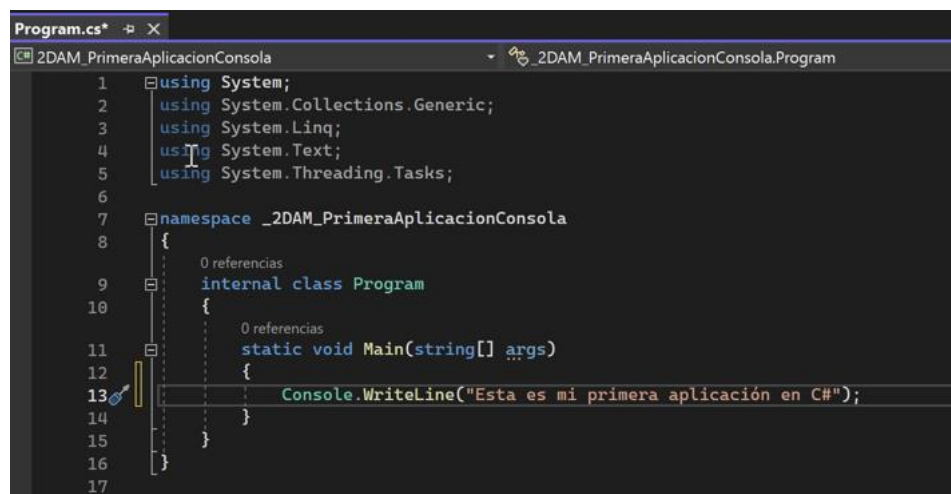
.NET Framework 4.7.2

3.1 HOLA MUNDO

Como en cualquier lenguaje, la forma típica de comenzar es realizar el primer programa “Hola mundo!”. En este caso no será distinto: con ello probaremos que todo funciona como debe y que el programa compila correctamente:



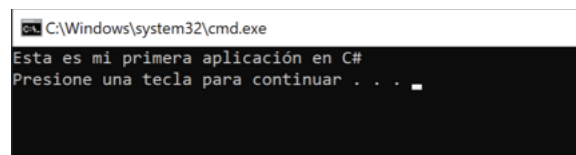
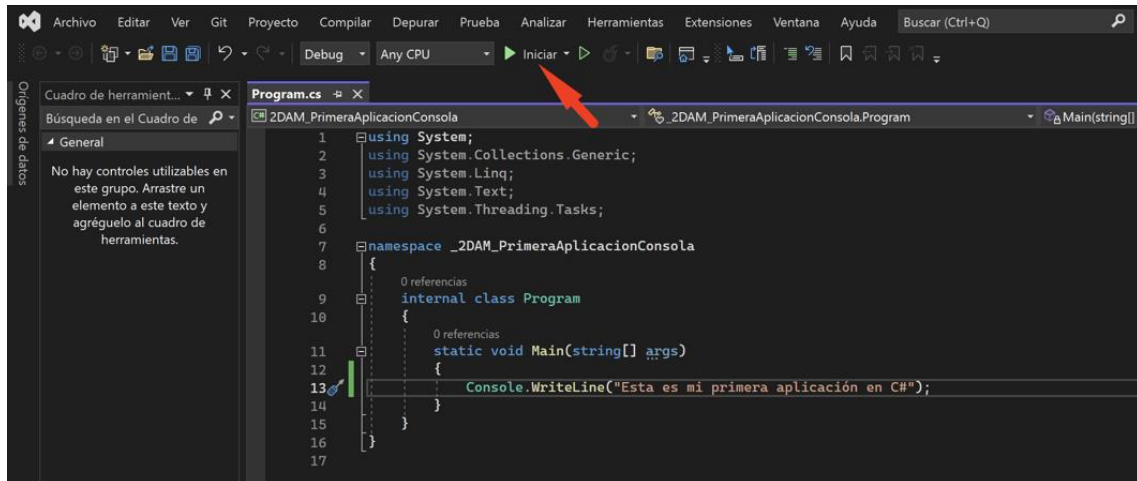
```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace _2DAM_PrimerAplicacionConsola
8 {
9     0 referencias
10     internal class Program
11     {
12         0 referencias
13         static void Main(string[] args)
14         {
15             Console.Write
16         }
17     }
```



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace _2DAM_PrimerAplicacionConsola
8 {
9     0 referencias
10     internal class Program
11     {
12         0 referencias
13         static void Main(string[] args)
14         {
15             Console.WriteLine("Esta es mi primera aplicación en C#");
16         }
17     }
```

UD 2. INTRODUCCIÓN A C#

Una vez generado el código, hay que hacer clic en “Iniciar” para compilar el programa:



3.2 SINTAXIS

A continuación procedemos a comentar la sintaxis del ejemplo anterior:

- Línea 1: “using System” significa que puede usar clases del espacio de nombres “System”
- Línea 7: “namespace” es usado para organizar código, y es un contenedor de clases y de otros namespaces
- Línea 8: las {} marcan el inicio y fin del bloque de código
- Línea 9: “class” es un contenedor para datos y métodos que proporcionan funcionalidad al programa. Cada línea de código en C# debe estar dentro de una clase. En este ejemplo, la clase se llama “Program”
- Línea 11: otro elemento que aparece siempre en un programa c# es el método Main. Todo el código dentro de las llaves será ejecutado.

- Línea 13: "Console" es una clase del espacio de nombres "System", y uno de sus métodos es "WriteLine()", el cual se encarga de mostrar texto por pantalla, en este caso "Hola mundo".

Si omitiéramos la línea "using System", tendríamos que utilizar "System.Console.WriteLine()" para obtener un resultado similar.

Al contrario que en Java, el nombre del fichero en C# no tiene por qué coincidir con el nombre de la clase, aunque normalmente sí coincide para una mejor organización. Los ficheros de código C# llevan la extensión .cs al final del nombre.

También es posible usar Console.Write("texto"). La diferencia es que "WriteLine" incluye un salto de línea después del texto.

4.ELEMENTOS DE UN PROGRAMA EN C#

4.1 COMENTARIOS

Como en cualquier lenguaje de programación, los comentarios son imprescindibles para explicar el código y hacerlo más legible.

En C# hay dos formas de poner comentarios. La primera de ellas es mediante "//". Todo el texto entre "//" y el final de la línea será ignorado:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _2DAM_PrimerAplicacionConsola
{
    internal class Program
    {
        static void Main(string[] args)
        {
            //esto es un comentario
            Console.WriteLine("Esta es mi primera aplicación en C#");
        }
    }
}
```

Si se desea crear varias líneas de comentarios hay que usar “/*” y “*/”. Todo texto entre estos dos símbolos será ignorado por C#. Por ejemplo:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _2DAM_PrimerAplicacionConsola
{
    internal class Program
    {
        static void Main(string[] args)
        {
            /*Esto es un comentario
             * escrito en varias líneas*/
            Console.WriteLine("Esta es mi primera aplicación en C#");
        }
    }
}
```

4.2 VARIABLES

Las variables son contenedores para almacenar diversos tipos de datos. Concretamente, en C#, hay diferentes tipos de variables:

int Almacena enteros, sin decimales

double Almacena números en coma flotante, con decimales

char Almacena caracteres simples, como ‘a’ o ‘B’. Los caracteres se encierran en comillas simples

string Almacena texto, como “Hola mundo”. Las cadenas se encierran en comillas dobles

bool Almacena valores con dos estados: verdadero (true) o falso (false)

Para crear una variable hay que especificar el tipo y asignarle un valor:

```
tipo nombreVariable = valor;
```

Por ejemplo:

```
namespace _2DAM_PrimerAplicacionConsola
{
    internal class Program
    {
        static void Main(string[] args)
        {
            /*Esto es un comentario
             * escrito en varias líneas*/
            string variable = "Uso de variables"
            Console.WriteLine(variable);
        }
    }
}
```

Otros ejemplos:

```
int miNumero = 13;
Console.WriteLine(miNumero);
```

que es equivalente a:

```
int miNumero;
miNumero = 13;
Console.WriteLine(miNumero);
```

4.2.1 CONSTANTES

Si quieres crear una constante, es decir, una variable pero cuyo valor no va a cambiar en el tiempo hay que usar la palabra reservada “const”:

```
const int miNumero = 15;
miNumero = 13;
Console.WriteLine(miNumero);
```

En este caso daría error de compilación: no se puede modificar el valor de una constante.

4.3 TIPOS DE DATOS

Los tipos de datos más comunes son los siguientes:

Data Type	Size	Description
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
bool	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter, surrounded by single quotes
string	2 bytes per character	Stores a sequence of characters, surrounded by double quotes

Un tipo de datos especifica el tamaño y el tipo de valores que almacena una variable. Es importante usar el tipo de datos correcto para cada variable para evitar errores, ahorrar tiempo y ahorrar memoria... Y, principalmente, para hacer el código más manipulable y legible.

4.4 ENTRADA DE DATOS

En apartados anteriores hemos visto que para mostrar un mensaje por consola se usa la directiva "Console.WriteLine". Ahora usaremos "Console.ReadLine()" para obtener datos de entrada por parte del usuario. En el siguiente ejemplo, el usuario puede introducir su nombre, que será almacenado en una variable llamada "userName". Y finalmente se imprimirá por pantalla su valor:

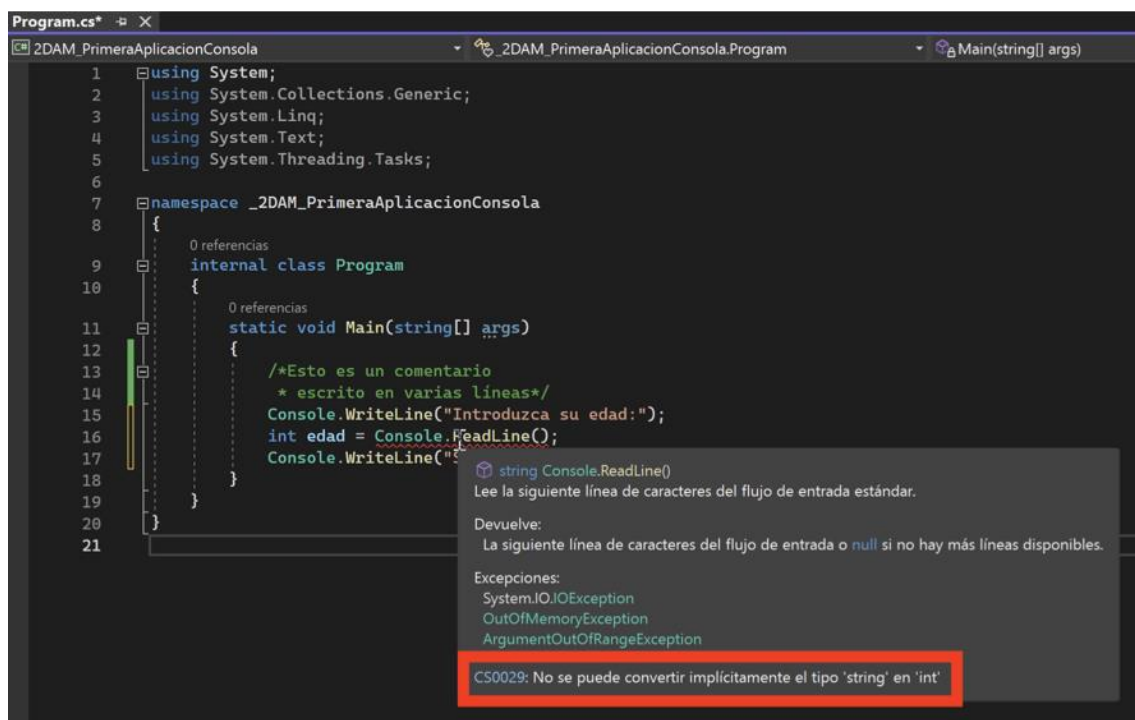
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _2DAM_PrimerAplicacionConsola
{
    internal class Program
    {
        static void Main(string[] args)
        {
            /*Esto es un comentario
```

UD 2. INTRODUCCIÓN A C#

```
* escrito en varias líneas*/
Console.WriteLine("Introduzca su nombre:");
string nombre = Console.ReadLine();
Console.WriteLine("Bienvenido al curso de 2DAM en Osuna, " + nombre);
}
}
```

El método “Console.ReadLine()” devuelve un string. Por lo tanto, no se puede obtener información de otro tipo de datos, como un entero. El siguiente programa mostraría un error:



El mensaje indica que no se puede convertir implícitamente de tipo String a int. Para solucionarlo, es necesario realizar un casting mediante el método “Convert.To”:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _2DAM_PrimerAplicacionConsola
{
    internal class Program
    {
```

UD 2. INTRODUCCIÓN A C#

```
static void Main(string[] args)
{
    /*Esto es un comentario
    * escrito en varias líneas*/
    Console.WriteLine("Introduzca su edad:");
    int edad = Convert.ToInt32(Console.ReadLine());
    Console.WriteLine("Su edad es " + edad);
}
}
```

4.5 OPERADORES

Los operadores se utilizan para realizar operaciones sobre variables y valores. En el siguiente ejemplo, usamos el operador + para sumar dos valores:

```
int suma = edad1 + edad2;
```

4.5.1 OPERADORES ARITMÉTICOS

Son los operadores usados para realizar operaciones matemáticas comunes:

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	x++
--	Decrement	Decreases the value of a variable by 1	x--

4.5.2 OPERADORES DE ASIGNACIÓN

Son los operadores utilizados para asignar valores a variables:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

4.5.3 OPERADORES DE COMPARACIÓN

Este tipo de operadores se utiliza para comparar dos valores:

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

4.5.4 OPERADORES LÓGICOS

Estos operadores son utilizados para determinar la lógica entre variables o valores:

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	<code>x < 5 && x < 10</code>
	Logical or	Returns true if one of the statements is true	<code>x < 5 x < 4</code>
!	Logical not	Reverse the result, returns false if the result is true	<code>!(x < 5 && x < 10)</code>

4.6 CLASE MATH

C# incluye varios métodos que permiten operaciones matemáticas sobre números. Por ejemplo:

`Math.Max(x,y)` Función que devuelve el máximo de los dos números

`Math.Min(x,y)` Función que devuelve el mínimo de los dos números

`Math.Sqrt(x)` Función que devuelve la raíz cuadrada del número

`Math.Abs(x)` Función que devuelve el valor absoluto del número

`Math.Round()` Función que redondea un número al valor más cercano

Ejemplo de uso:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace PruebaMath
{
    internal class Program
    {
        static void Main(string[] args)
        {
            /*Esto es un comentario
             * escrito en varias líneas*/
        }
    }
}
```

UD 2. INTRODUCCIÓN A C#

```
int numero1, numero2, numero3;
double numero4;
numero1 = 5;
numero2 = 64;
numero3 = -15;
numero4 = 9.99;
Console.WriteLine("El máximo de los dos números es: " + Math.Max(numero1, numero2));
Console.WriteLine("El mínimo de los dos números es: " + Math.Min(numero1, numero2));
Console.WriteLine("La raíz cuadrada del segundo número es: " + Math.Sqrt(numero2));
Console.WriteLine("El valor absoluto del tercer número es: " + Math.Abs(numero3));
Console.WriteLine("El valor aproximado de numero4 es: " + Math.Round(numero4));

Console.ReadLine();
}
}
}
```

4.7 STRINGS

Un String en C# es un objeto que contiene propiedades y métodos que pueden ejecutar ciertas operaciones sobre las cadenas. Por ejemplo, la longitud de una cadena puede ser obtenida con la propiedad "Length":

```
string texto = "Hola mundo";
Console.WriteLine("La longitud del string es " + texto.Length);
```

Existen otras muchas funciones: "ToUpper()", "ToLower()"...

También es posible concatenar cadenas usando para ello el operador "+" o el método "Concat()"

4.8 BOOLEANOS

En programación frecuentemente es necesario usar un tipo de datos que solo permita dos valores, como Sí/No, Encendido/Apagado, True/False. Para ello, C# tiene el tipo de datos "bool", que puede tener los valores "true" o "false"

```
class Program
{
    static void Main(string[] args)
```

UD 2. INTRODUCCIÓN A C#

```
{  
    bool esCSharpDivertido = true;  
    bool esCSharpAburrido = false;  
    Console.WriteLine(esCSharpDivertido);  
    Console.WriteLine(esCSharpAburrido);  
    Console.ReadLine();  
}  
}
```

También se puede utilizar un operador de comparación, como el ">" para comprobar si una expresión (o variable) es "true":

```
class Program  
{  
    static void Main(string[] args)  
    {  
        int x = 10;  
        int y = 9;  
        Console.WriteLine(x > y); // devuelve True, porque 10 es mayor que 9  
    }  
}
```

4.9 FORMATO DE SALIDA

Como ya se ha comentado y visto en este curso, para mostrar una variable por pantalla se utiliza el método "Console.WriteLine". Además, es posible usar modificadores de formato para representar la salida en el formato deseado:

```
// Format a negative integer or floating-point number in various ways.  
Console.WriteLine("Standard Numeric Format Specifiers");  
Console.WriteLine(  
    "(C) Currency: . . . . . {0:C}\n" +  
    "(D) Decimal: . . . . . {0:D}\n" +  
    "(E) Scientific: . . . . . {1:E}\n" +  
    "(F) Fixed point: . . . . . {1:F}\n" +  
    "(G) General: . . . . . {0:G}\n" +  
    "    (default): . . . . . {0} (default = 'G')\n" +  
    "(N) Number: . . . . . {0:N}\n" +  
    "(P) Percent: . . . . . {1:P}\n" +  
    "(R) Round-trip: . . . . . {1:R}\n" +  
    "(X) Hexadecimal: . . . . . {0:X}\n",  
    -123, -123.45f);
```

```
// Format the current date in various ways.
Console.WriteLine("Standard DateTime Format Specifiers");
Console.WriteLine(
    "(d) Short date: . . . . . {0:d}\n" +
    "(D) Long date: . . . . . {0:D}\n" +
    "(t) Short time: . . . . . {0:t}\n" +
    "(T) Long time: . . . . . {0:T}\n" +
    "(f) Full date/short time: . . {0:f}\n" +
    "(F) Full date/long time: . . {0:F}\n" +
    "(g) General date/short time: . {0:g}\n" +
    "(G) General date/long time: . {0:G}\n" +
    " (default): . . . . . {0} (default = 'G')\n" +
    "(M) Month: . . . . . {0:M}\n" +
    "(R) RFC1123: . . . . . {0:R}\n" +
    "(s) Sortable: . . . . . {0:s}\n" +
    "(u) Universal sortable: . . . {0:u} (invariant)\n" +
    "(U) Universal full date/time: {0:U}\n" +
    "(Y) Year: . . . . . {0:Y}\n",
    thisDate);
```

```
// Format a Color enumeration value in various ways.
Console.WriteLine("Standard Enumeration Format Specifiers");
Console.WriteLine(
    "(G) General: . . . . . {0:G}\n" +
    " (default): . . . . . {0} (default = 'G')\n" +
    "(F) Flags: . . . . . {0:F} (flags or integer)\n" +
    "(D) Decimal number: . . . . {0:D}\n" +
    "(X) Hexadecimal: . . . . . {0:X}\n",
    Color.Green);
```

4.10 ARRAYS

Los arrays son utilizados para almacenar múltiples valores en una variable simple, en vez de declarar variables separadas para cada valor. Para declarar un array, define el tipo de variable junto a los corchetes:

```
class Program
{
    static void Main(string[] args)
    {
        string[] coches = { "Ford", "Seat", "Audi" };
        Console.WriteLine(coches[0]);
        Console.ReadLine();
    }
}
```

En este ejemplo se le ha asignado tres valores al array, con los nombres de las marcas de coches. A continuación se ha realizado la llamada al primer elemento de dicho array, para lo que hay que utilizar la nomenclatura `nombre_array[posición]`, donde la primera posición del array es la 0.

Para cambiar el valor de alguna posición del array, bastaría con acceder a su posición con el operador =:

```
cars[0] = "Opel";
```

Existen varias propiedades asociadas a un array, como la que indica su longitud (array.Length).

Para recorrer un array, la mejor opción es utilizar alguna estructura iterativa como un bucle foreach. Su sintaxis es la siguiente:

```
foreach (type variableName in arrayName)
{
    // code block to be executed
}
```

Existen métodos para ordenar arrays:

```
Array.Sort(cars);
```

También existen métodos en el namespace System.Linq para realizar operaciones con arrays:

```
cars.Max()
cars.Min()
cars.Sum()
```

5. ESTRUCTURAS ITERATIVAS

No todos los problemas pueden resolverse empleando estructuras secuenciales. Cuando hay que tomar una decisión aparecen las estructuras condicionales.

En nuestra vida diaria se nos presentan situaciones donde debemos decidir. ¿Elijo la carrera A o la carrera B? ¿Me pongo este pantalón? Para ir al trabajo, ¿elijo el camino A o el camino B? Al cursar una carrera, ¿elijo el turno mañana, tarde o noche?

Por supuesto que en un problema se combinan estructuras secuenciales y condicionales. En este apartado veremos las distintas estructuras iterativas que proporciona C#.

5.1 IF...ELSE

C# admite las condiciones lógicas habituales de las matemáticas:

- Menor que: $a < b$
- Menor o igual a: $a \leq b$
- Mayor que: $a > b$
- Mayor o igual a: $a \geq b$
- Igual a: $a == b$
- No es igual a: $a != b$

Puedes utilizar estas condiciones para realizar diferentes acciones para diferentes decisiones.

C# tiene las siguientes declaraciones condicionales:

- if para especificar un bloque de código que se ejecutará, si una condición especificada es verdadera
- else para especificar un bloque de código que se ejecutará, si la misma condición es falsa
- else if para especificar una nueva condición para probar, si la primera condición es falsa

- switch para especificar varias opciones alternativas de código que se ejecutarán en función de lo que se seleccione

La sintaxis de if es la siguiente:

```
if (condition)
{
    // bloque de Código a ejecutar si la condición es true
}
```

En muchas ocasiones será necesario también incluir código que queremos que se ejecute cuando la condición sea falsa. En ese caso, hay que usar el else:

```
if (condition)
{
    // bloque de Código a ejecutar si la condición es true
}
else
{
    // bloque de Código a ejecutar si la condición es false
}
```

A continuación se muestra un ejemplo de uso del if..else:

```
static void Main(string[] args)
{
    int hora = 13;
    if (hora < 12)
    {
        Console.WriteLine("Buenos días");
    }
    else
    {
        Console.WriteLine("Buenas tardes");
    }
}
```

Hay otra variante de la estructura if..else, y es la siguiente:

```
if (condition1)
{
    // bloque de Código a ejecutar si la condición 1 es true
}
else if (condition2)
{
    // bloque de Código a ejecutar si la condición 1 es false y la condición 2 es true
}
else
{
    // bloque de Código a ejecutar si la condición 1 es false y la condición 2 también
}
```

Hay una variante compacta del if, que se conoce como “operador ternario” porque consta de tres operandos. A menudo se utiliza para reemplazar declaraciones if..else simples. Su sintaxis es:

```
variable = (condition) ? expressionTrue : expressionFalse;
```

El ejemplo anterior se podría reescribir así:

```
static void Main(string[] args)
{
    int hora = 13;
    string resultado = (hora < 12) ? "Buenos días" : "Buenas tardes";
    Console.WriteLine(resultado);
}
```

5.2 SWITCH

Este tipo de estructura se utiliza para ejecutar determinado código dependiendo de una condición. La sintaxis es la siguiente:

```
switch(expression)
{
    case x:
        // código si expression vale x
        break;
    case y:
        // código si expression vale y
        break;
    default:
```

```
// código por defecto  
break;  
}
```

El funcionamiento es el siguiente:

- La expresión es evaluada una vez.
- La expresión es comparada con los valores de cada “case”
- Si hay alguna coincidencia, se ejecuta el bloque correspondiente
- Si no hay ninguna coincidencia, se ejecuta el bloque de código de la sección “default”
- “break” implica que se sale del bloque “switch”
- “default” es opcional, e indica el código a ejecutar si no hay ninguna coincidencia

Un ejemplo de uso es el siguiente:

```
static void Main(string[] args)  
{  
    int dia = 4;  
    switch (dia)  
    {  
        case 1:  
            Console.WriteLine("Lunes");  
            break;  
        case 2:  
            Console.WriteLine("Martes");  
            break;  
        case 3:  
            Console.WriteLine("Miércoles");  
            break;  
        case 4:  
            Console.WriteLine("Jueves");  
            break;  
        case 5:  
            Console.WriteLine("Viernes");  
            break;  
        case 6:  
            Console.WriteLine("Sábado");  
            break;  
        case 7:  
            Console.WriteLine("Domingo");  
            break;  
    }  
    // Muestra por pantalla "Jueves" (día 4)  
}
```

5.3 WHILE

Los bucles pueden ejecutar un determinado código varias veces hasta que se alcance una determinada condición. Los bucles ahorran tiempo, reduce errores y hacen el código más eficaz. Dentro del tipo de bucles “While”, se pueden distinguir dos tipos:

5.3.1 BUCLE WHILE

Mediante este tipo de bucle se ejecuta el código especificado hasta que la condición es “true”. Su sintaxis es la siguiente:

```
while (condicion)
{
    // Código a ejecutar
}
```

El siguiente ejemplo usa un bucle While para imprimir por pantalla los números menores de 5:

```
static void Main(string[] args)
{
    int i = 0;
    while (i < 5)
    {
        Console.WriteLine(i);
        i++;
    }
}
```

5.3.2 BUCLE DO/WHILE

En este tipo de bucle el código se ejecuta como mínimo una vez. La sintaxis es la siguiente:

```
do
{
    // code block to be executed
}
while (condition);
```

En este ejemplo, el resultado es el mismo que con el uso del bucle “while”:

```
static void Main(string[] args)
{
    int i = 0;
    do
    {
        Console.WriteLine(i);
        i++;
    }
    while (i < 5);
}
```

5.4 FOR

Este tipo de bucles se utiliza cuando se conoce previamente el número exacto de veces que se desea ejecutar el bucle. En estos casos, es preferible usar el “for”. La sintaxis es la siguiente:

```
for (statement 1; statement 2; statement 3)
{
    // bloque de Código para ser ejecutado
}
```

Statement 1 es ejecutado (una sola vez) antes de comenzar la ejecución del bloque de código.

Statement 2 define la condición para que se ejecute el bloque de código.

Statement 3 es ejecutado (cada vez) tras cada iteración por el bloque de código.

En el siguiente ejemplo se muestra por pantalla los números entre el 0 y el 4:

```
static void Main(string[] args)
{
    for (int i = 0; i < 5; i++)
    {
        Console.WriteLine(i);
    }
}
```

5.5 BREAK Y CONTINUE

La declaración “break” se puede utilizar para salir abruptamente de un bucle. En el siguiente ejemplo sale del bucle cuando i es igual a 4:

```
static void Main(string[] args)
{
    for (int i = 0; i < 10; i++)
    {
        if (i==4)
        {
            break;
        }
        Console.WriteLine(i);
    }
}
```

Hay otra sentencia relacionada, como es “continue”, pero que en vez de salir totalmente del bucle, lo que hace es “saltar” esa iteración. En este ejemplo se aprecia perfectamente:

```
static void Main(string[] args)
{
    for (int i = 0; i < 10; i++)
    {
        if (i==4)
        {
            continue;
        }
        Console.WriteLine(i);
    }
    Console.ReadLine();
}
```

6.MÉTODOS

Un método es un bloque de código que solo es ejecutado cuando es llamado. A los métodos se les puede pasar datos mediante los parámetros. Los métodos también se conocen como “funciones”. El uso de métodos permite reutilizar código.

6.1 CREACIÓN DE MÉTODOS

Un método es definido por el nombre del método, seguido por paréntesis. C# provee métodos predefinidos, como el Main(), pero es posible crear nuestros propios métodos. La sintaxis para la definición de un método es la siguiente:

```
class Program
{
    static void MiMetodo()
    {
        // Código a ejecutarse
    }
}
```

MiMetodo es el nombre del método.

“static” significa que el método pertenece a la clase “Program” y no a un objeto de la clase “Program”.

void significa que este método no devuelve ningún valor.

6.2 LLAMADA A MÉTODOS

Para llamar a un método basta con escribir el nombre del método seguido de dos paréntesis y un ;. Este es un ejemplo:

```
class Program
{
    static void MiMetodo()
    {
        Console.WriteLine("Mi primer método");
    }
}
```

```
static void Main(string[] args)
{
    MiMetodo();
    Console.ReadKey();
}
```

6.3 PARÁMETROS

Los métodos pueden recibir información a través de los parámetros. Los parámetros actúan como variables dentro del método. Estos parámetros se especifican después del nombre del método, dentro de los paréntesis. Se pueden añadir tantos parámetros como sean necesarios, separados por “coma”.

En el siguiente ejemplo se utiliza un método con un argumento (string):

```
class Program
{
    static void MiMetodo(string nombre)
    {
        Console.WriteLine("Mi nombre es {0}", nombre);
    }
    static void Main(string[] args)
    {
        MiMetodo("Dani");
        Console.ReadKey();
    }
}
```

Es posible definir un argumento con un valor por defecto. El siguiente ejemplo mostraría por pantalla dos líneas idénticas con el valor “Mi nombre de Dani”:

```
class Program
{
    static void MiMetodo(string nombre="Dani")
    {
        Console.WriteLine("Mi nombre es {0}", nombre);
    }
    static void Main(string[] args)
    {
        MiMetodo("Dani");
        MiMetodo();
        Console.ReadKey();
    }
}
```

```
}
}
```

Por supuesto, también es posible crear un método con múltiples parámetros. La llamada se realizaría incluyendo los argumentos dentro del paréntesis, separados por “comas”.

En los ejemplos anteriores, el método no ha devuelto ningún valor, simplemente ha realizado una acción determinada. Si deseamos que el método devuelva algún valor, hay que utilizar “return”. En el siguiente ejemplo se hace uso de ello:

```
class Program
{
    1 referencia
    static int MiMetodo(int x)
    {
        return 5 + x;
    }
    0 referencias
    static void Main(string[] args)
    {
        Console.WriteLine(MiMetodo(3));
        Console.ReadKey();
    }
}
```

Por último: cuando se realiza una llamada con argumentos a un método, estos se asignan de forma ordenada: el primer parámetro corresponde al primero del método, y así sucesivamente. Pero también existe la posibilidad de asignarlo en otro orden, indicando el nombre de cada argumento. Por ejemplo:

```
MiMetodo(parametro3: "Alumno1", parametro2: "Alumno2", parametro3: "Alumno3");
```

6.4 SOBRECARGA DE MÉTODOS

Con la sobrecarga de métodos, múltiples métodos pueden tener el mismo nombre pero con diferentes parámetros. Ejemplo:

```
int MiMetodo(int x)
float MiMetodo(float x)
double MiMetodo(double x, double y)
```

A continuación aparece un ejemplo real del uso de métodos sobrecargados:

```
class Program
{
    static int Suma(int x, int y)
    {
        return x + y;
    }

    static double Suma(double x, double y)
    {
        return x + y;
    }
    static void Main(string[] args)
    {
        int miNumero = Suma(3, 7);
        double miNumero2 = Suma(4.65, 3.67);
        Console.WriteLine("Int: {0}", miNumero);
        Console.WriteLine("Double: {0}", miNumero2);
        Console.ReadKey();
    }
}
```

7.CLASES

De forma complementaria a la programación con funciones o procedimientos aparece la programación orientada a objetos, a partir de la cual se crean instancias de clases, los objetos, que contienen tanto datos como métodos. La programación orientada a objetos tiene las siguientes ventajas:

- Es más rápida y fácil de ejecutar.
- Ayuda a mantener el código fácil de mantener, modificar y depurar.
- Hace posible crear aplicaciones completamente reutilizables con menos código y menos tiempo de desarrollo.

Todo en C# está asociado con clases y objetos, junto con sus atributos y métodos. Por ejemplo: en la vida real, un automóvil es un objeto. El automóvil tiene atributos, como peso y color, y métodos, como conducir y frenar. Una clase es como un constructor de objetos o un “plano” para crear objetos.

7.1 CREAR UNA CLASE

Para crear una clase hay que usar la palabra clave “class”. La sintaxis es la siguiente:

```
class Coche
{
    string color = "rojo";
}
```

Es una buena práctica (aunque no obligatorio) comenzar con la primera letra en mayúsculas al nombrar clases. Además, es común que el nombre del archivo C# y la clase coincidan, ya que hace que el código esté organizado (no es obligatorio, al contrario de lo que sucede en Java).

7.2 CREAR UN OBJETO

Un objeto se crea a partir de una clase. En el apartado anterior se ha creado la clase nombrada como “Coche”, por lo que ahora se puede utilizar para crear objetos. Para crear un objeto de “Coche” hay que especificar el nombre de la clase, seguido del nombre del objeto y usando la palabra clave “new”. A continuación un ejemplo:

```
class Coche
{
    string color = "rojo";
}
```

```
static void Main(string[] args)
{
    Coche myObj = new Coche();
    Console.WriteLine(myObj.color);
}
```

Para acceder a variables/campos dentro de una clase se utiliza el ".".

Se puede crear varios objetos de una misma clase:

```
class Coche
{
    string color = "rojo";
    static void Main(string[] args)
    {
        Coche myObj1 = new Coche();
        Coche myObj2 = new Coche();
        Console.WriteLine(myObj1.color);
        Console.WriteLine(myObj2.color);
    }
}
```

7.3 USO DE MÚLTIPLES CLASES

También es posible crear un objeto de una clase y acceder a él en otra clase. Esto se usa a menudo para una mejor organización de clases: una clase tiene todos los campos y métodos, mientras que la otra clase contiene el Main(). A continuación se muestra un ejemplo:

programa1.cs

```
class Coche
{
    public string color = "rojo";
}
```

programa2.cs

```
class Program
{
    static void Main(string[] args)
    {
        Coche myObj = new Coche();
        Console.WriteLine(myObj.color);
    }
}
```

7.4 MIEMBROS DE UNA CLASE

Los campos y métodos dentro de las clases a menudo se denominan "miembros de clase". En el siguiente ejemplo se crea una clase "Coche" con tres miembros: dos campos y un método:

```
class MiClase
{
    // Miembros de la clase
    string color = "rojo";    // campo
    int velMax = 200;        // campo
    public void acelera()    // método
    {
        Console.WriteLine("El coche está acelerando!");
    }
}
```

Para acceder a los campos se utiliza el ".". Por ejemplo:

```
class Coche
{
    string color = "rojo";
    int velMax = 200;

    static void Main(string[] args)
    {
        Coche myObj = new Coche();
        Console.WriteLine(myObj.color);
        Console.WriteLine(myObj.velMax);
    }
}
```

También se puede dejar los campos en blanco y modificarlos al crear el objeto:

```
class Coche
{
    string color;
    int velMax;

    static void Main(string[] args)
    {
        Coche myObj = new Coche();
        myObj.color = "rojo";
        myObj.velMax = 200;
        Console.WriteLine(myObj.color);
        Console.WriteLine(myObj.velMax);
    }
}
```

Los métodos normalmente pertenecen a una clase y definen cómo se comporta un objeto de una clase. Al igual que con los campos, se puede acceder a métodos con la sintaxis de puntos. Además, hay que usar el nombre del método seguido de dos paréntesis y un punto y coma para ejecutar el método:

```
class Coche
{
    string color;           // campo
    int velMax;             // campo
    public void acelera()   // método
    {
        Console.WriteLine("El coche está acelerando!");
    }

    static void Main(string[] args)
    {
        Coche myObj = new Coche();
        myObj.acelera();    // Llama al método
    }
}
```

7.5 CONSTRUCTORES

Un constructor es un método especial que se utiliza para inicializar objetos. La ventaja de un constructor es que se llama cuando se crea un objeto de una clase. Se puede utilizar para establecer valores iniciales para los campos:

```
class Coche
{
    public string model; // Crea un campo
    // Crea un constructor para la clase Coche
    public Coche()
    {
        model = "Ford"; // Configura el valor de la variable model
    }
    static void Main(string[] args)
    {
        Coche miCoche = new Coche(); // Crea un objeto de la clase Coche
        Console.WriteLine(miCoche.model); // Imprime la variable model
    }
}
```

Los constructores también pueden tomar parámetros, que se utilizan para inicializar campos. En el siguiente ejemplo se agrega un string como parámetro al constructor. Dentro del constructor se asigna al campo model el valor del parámetro modelName. Cuando se llama al constructor, pasamos un parámetro al constructor ("Mustang"), que establecerá el valor de model a "Mustang":

```
class Coche
{
    public string model;

    // Create a class constructor with a parameter
    public Coche(string modelName)
    {
        model = modelName;
    }

    static void Main(string[] args)
    {
        Coche miCoche = new Coche("Mustang");
        Console.WriteLine(miCoche.model);
    }
}
```

7.6 MODIFICADORES DE ACCESO

En C# existen los siguientes modificadores de acceso:

Modifier	Description
public	The code is accessible for all classes
private	The code is only accessible within the same class
protected	The code is accessible within the same class, or in a class that is inherited from that class. You will learn more about inheritance in a later chapter
internal	The code is only accessible within its own assembly, but not from another assembly. You will learn more about this in a later chapter

MODIFICADOR PRIVADO

Si declaras un campo con un modificador “private”, solo se puede acceder desde dentro de la misma clase. Por ejemplo:

```
namespace ConsoleApp13
{
    class Coche
    {
        private string model = "Audi";

        static void Main(string[] args)
        {
            Coche myObj = new Coche();
            Console.WriteLine(myObj.model);
            Console.ReadKey();
        }
    }
}
```

Si se intentara acceder al atributo “model” desde fuera de la clase “Coche”, se produciría un error.

MODIFICADOR PÚBLICO

Si se declara un campo con un “public” como modificador de acceso, será accesible para todas las clases. Por ejemplo:

```

namespace ConsoleApp13
{
    class Coche
    {
        public string model = "Seat";
    }

    class Program
    {
        static void Main(string[] args)
        {
            Coche myObj = new Coche();
            Console.WriteLine(myObj.model);
            Console.ReadKey();
        }
    }
}

```

De forma predeterminada, todos los miembros de una clase son “private” si no se especifica un modificador de acceso

7.7 PROPIEDADES

En un contexto de “Encapsulación” se definen parte de datos con información “sensible”, en cuyo caso se declaran estos campos/variables como “private”. Como vimos en el apartado anterior, solo serían accesibles desde dentro de la clase. En ciertas ocasiones podemos necesitar acceder a esos datos, lo que puede realizarse a través de las propiedades, que es un término medio entre variables y métodos, y tiene dos métodos: get y set. Por ejemplo:

```

static void Main(string[] args)
{
    class Person
    {
        private string name; // field

        public string Name // property
        {
            get { return name; } // get method
            set { name = value; } // set method
        }
    }
}

```

La propiedad “Name” se asocia con el campo “name”. Es una buena práctica usar el mismo nombre para ambos, pero con la primera letra de la propiedad en mayúsculas.

El método “get” devuelve el valor de la variable “name”.

El método “set” asigna “value” a la variable “name”.

En el siguiente ejemplo puedes ver un ejemplo de uso:

```
namespace ConsoleApp15
{
    class Person
    {
        private string name; // field
        public string Name // property
        {
            get { return name; }
            set { name = value; }
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Person myObj = new Person();
            myObj.Name = "Liam";
            Console.WriteLine(myObj.Name);
        }
    }
}
```

Para más información: <https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/using-properties>

7.8 HERENCIA

En C# es posible heredar campos y métodos de una clase (clase base) a otra (clase derivada). Para heredar de una clase, se usa el símbolo “:”. En el siguiente ejemplo se puede ver:

```

class Vehiculo // clase base (parent)
{
    public string marca = "Ford"; // campo "marca" de Vehículo
    public void acelera() // método "acelera" de Vehículo
    {
        Console.WriteLine("Acelerando!");
    }
}

class Coche : Vehiculo // clase derivada (child)
{
    public string modelo = "Mustang"; // campo "modelo" de la clase Coche
}

class Program
{
    static void Main(string[] args)
    {
        // Crea un objeto Coche
        Coche myCar = new Coche();

        // Llama al método acelera() de la clase Vehiculo
        myCar.acelera();

        // Muestra el valor del campo marca de la clase Vehiculo y el valor de modelo de la clase Coche
        Console.WriteLine(myCar.marca + " " + myCar.modelo);
        Console.ReadKey();
    }
}

```

Si no deseamos que de una clase puedan heredar otras, hay que ponerle a la clase base la palabra reservada “sealed”.

7.9 POLIMORFISMO

El polimorfismo ocurre cuando tenemos varias clases relacionadas con otra por herencia, y se produce cuando en la clase heredada tenemos nuestra propia implantación de algún método de la clase base. Por ejemplo:

```

class Animal // Clase base (parent)
{
    public void sonidoAnimal()
    {
        Console.WriteLine("El animal hace un sonido");
    }
}

class Cerdo : Animal // Clase derivada (child)

```

```

{
    public void sonidoAnimal()
    {
        Console.WriteLine("El cerdo hace: wee wee");
    }
}

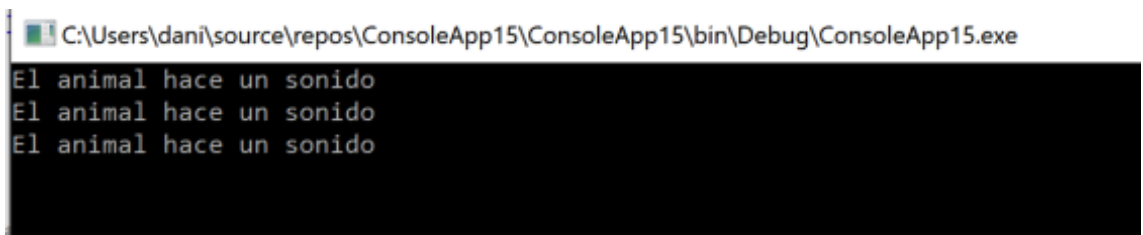
class Perro : Animal // Clase derivada (child)
{
    public void sonidoAnimal()
    {
        Console.WriteLine("El perro hace: guau guau");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Animal miAnimal = new Animal(); // Crea un objeto Animal
        Animal miCerdo = new Cerdo(); // Crea un objeto Cerdo
        Animal miPerro = new Perro(); // Crea un objeto Perro

        miAnimal.sonidoAnimal();
        miCerdo.sonidoAnimal();
        miPerro.sonidoAnimal();
        Console.ReadKey();
    }
}

```

La ejecución es la siguiente:



```

C:\Users\dani\source\repos\ConsoleApp15\ConsoleApp15\bin\Debug\ConsoleApp15.exe
El animal hace un sonido
El animal hace un sonido
El animal hace un sonido

```

El resultado no es el esperado. ¿Por qué? Porque el método de la clase base sobrescribe al método de la clase derivada cuando comparten el mismo nombre. C#

provee una forma de sobrescribir el método de la clase base añadiendo la palabra clave “virtual” en la definición del método en la case base, y usando la palabra clave “override” para el método en cada clase derivada:

```
class Animal // Clase base (parent)
{
    public virtual void sonidoAnimal()
    {
        Console.WriteLine("El animal hace un sonido");
    }
}

class Cerdo : Animal // Clase derivada (child)
{
    public override void sonidoAnimal()
    {
        Console.WriteLine("El cerdo hace: wee wee");
    }
}

class Perro : Animal // Clase derivada (child)
{
    public override void sonidoAnimal()
    {
        Console.WriteLine("El perro hace: guau guau");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Animal miAnimal = new Animal(); // Crea un objeto Animal
        Animal miCerdo = new Cerdo(); // Crea un objeto Cerdo
        Animal miPerro = new Perro(); // Crea un objeto Perro

        miAnimal.sonidoAnimal();
        miCerdo.sonidoAnimal();
        miPerro.sonidoAnimal();
        Console.ReadKey();
    }
}
```

La ejecución es la siguiente:

```
C:\Users\dani\source\repos\ConsoleApp15\ConsoleApp15\bin\Debug\ConsoleApp15.exe
El animal hace un sonido
El cerdo hace: wee wee
El perro hace: guau guau
-
```

7.10 ARCHIVOS

Para trabajar con ficheros se usa la clase “File” del namespace “System.IO”. Puedes consultar el detalle al completo de esta clase en la documentación oficial de Microsoft:

<https://docs.microsoft.com/en-us/dotnet/api/system.io.file?view=netframework-4.7.2>

En el siguiente ejemplo puedes ver cómo se puede trabajar con ficheros, almacenando en ellos información y/o recuperándola:

```
static void Main(string[] args)
{
    string path = @"c:\Users\dani\fichero.txt";

    //Comprueba si existe el fichero
    if (!File.Exists(path))
    {
        // Si no existe, lo crea
        string createText = "Bienvenidos a 2º de DAM" + Environment.NewLine;
        File.WriteAllText(path, createText);
    }

    // Agrega una nueva línea en el fichero de texto
    string appendText = "Mucha suerte!!" + Environment.NewLine;
    File.AppendAllText(path, appendText);

    // Abre el fichero para leer
    string readText = File.ReadAllText(path);
    Console.WriteLine(readText);
    Console.ReadKey();
}
```

7.11 EXCEPCIONES

Cuando se ejecuta código en C# pueden ocurrir diferentes errores: provocados por el programador, debidos a una entrada de datos incorrecta, u otro tipo de errores con efectos indeseados. Cuando ocurre un error, C# normalmente terminará el programa y generará un mensaje de error: habrá “lanzado” una excepción.

Para gestionar esos errores, se puede definir el código a proteger dentro de un `try..catch`, por ejemplo:

```
static void Main(string[] args)
{
    try
    {
        int[] myNumbers = { 1, 2, 3 };
        Console.WriteLine(myNumbers[10]);
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
    }
    Console.ReadKey();
}
```

El resultado será el siguiente:

